

Hierarchical Long-Term Semantic Memory for LinkedIn’s Hiring Agent

Zhentao Xu
zhexu@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Shangjin Zhang
shanzhang@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Emirhan Poyraz
epoyraz@linkedin.com
LinkedIn Corporation
New York, NY, USA

Yvonne Li
yiwli@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Ye Jin
yejin@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Xie Lu
xilu@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Xiaoyang Gu
xgu@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Karthik Ramgopal
kramgopa@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Praveen Kumar Bodigutla
pbodigutla@linkedin.com
LinkedIn Corporation
Sunnyvale, CA, USA

Xiaofeng Wang
xiaofwang@linkedin.com
LinkedIn Corporation
New York, NY, USA

Abstract

Large Language Model (LLM) agents are increasingly used in real-world products, where personalized and context-aware user interactions are essential. A central enabler of such capabilities is the agent’s long-term semantic memory system, which extracts implicit and explicit signals from noisy longitudinal behavioral data, stores them in a structured form, and supports low-latency retrieval. Building industrial-grade long-term memory for LLM agents raises five challenges: scalability, low-latency retrieval, privacy constraints, adaptability, and observability. We introduce the *Hierarchical Long-Term Semantic Memory* (HLTM) framework, which organizes textual data into a schema-aligned memory tree that captures semantic knowledge at multiple levels of granularity, enabling scalable ingestion, privacy-aware storage, low-latency retrieval, and transparent provenance; HLTM further incorporates an adaptation mechanism to generalize across diverse use cases. Extensive evaluations on LinkedIn’s Hiring Assistant show that HLTM improves answer correctness by more than 5% and retrieval F1 by more than 10%, while significantly advancing the Pareto frontier between query and indexing latency. HLTM has been fully deployed in LinkedIn’s Hiring Assistant to power core personalization features in production hiring workflows.

CCS Concepts

• **Computing methodologies** → **Natural language processing; Knowledge representation and reasoning**; • **Information systems** → **Information retrieval; Data management systems**.

Keywords

agent memory, hierarchical memory, LLM agents, production LLM systems, retrieval-augmented generation, AI for recruiting

ACM Reference Format:

Zhentao Xu, Shangjin Zhang, Emirhan Poyraz, Yvonne Li, Ye Jin, Xie Lu, Xiaoyang Gu, Karthik Ramgopal, Praveen Kumar Bodigutla, and Xiaofeng Wang. 2026. Hierarchical Long-Term Semantic Memory for LinkedIn’s Hiring Agent. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3818432>

1 Introduction

Large language models (LLMs) such as GPT [1], Claude [2], and Gemini [5] have demonstrated strong performance across a wide range of tasks in real-world agent systems, including question answering, code generation, and retrieval-augmented search [11, 28, 34]. Modern LLM-based agents [23] augment these models with external tools [20] and carefully designed workflows to enhance their reasoning and planning capabilities [24, 27]. For example, LinkedIn’s Hiring Assistant [8] is an AI recruiting agent that helps recruiters source, evaluate, and outreach to candidates at scale. Agentic memory systems [4, 10, 26, 30, 31, 33] support an agent’s long-term interaction with its environment by persisting user-specific signals from historical data and leveraging them to personalize future agent–user exchanges. For instance, LinkedIn’s



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
KDD 2026, Jeju Island, Republic of Korea.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3818432>

Hiring Assistant [8] needs to extract personalized hiring preferences from recruiters’ past projects and activities, and recall this recruiter-specific memory to generate tailored responses in future interactions. Achieving this personalization at scale requires a memory system that reliably extracts implicit and explicit signals from noisy longitudinal data, stores them in structured form, and retrieves them efficiently during real-time conversations.

Recent work has advanced agents’ long-context capabilities by introducing explicit memory modules. Systems such as MemGPT, Mem0, MemoryBank, ReadAgent, MemOS, SimpleMem, and SCM [4, 12, 14, 15, 17, 22, 33] extend LLMs with external stores that swap, persist, and selectively retrieve interaction traces beyond the native context window. Complementary efforts explore more structured representations: GraphRAG, HippoRAG, A-Mem, and G-Memory [6, 10, 26, 30] organize knowledge into graphs to support multi-hop reasoning, while RAPTOR, TreeRAG, and MemTree [18, 19, 21] exploit hierarchical structure for scalable long-context retrieval. Despite these advances, designing long-term semantic memory for large-scale, industrial-grade agents remains challenging across several key dimensions:

- **Challenge 1: Scalability** A production memory system must ingest and process large-scale structured or unstructured data from diverse sources and formats. This demands compact memory representations, highly parallelizable ingestion pipelines, and lossless incremental updates.
- **Challenge 2: Low-latency Retrieval** Many scenarios impose strict latency constraints. For example, chat-based agents must respond within tight budgets to maintain a fluent conversation, leaving little room for expensive retrieval or oversized context windows at serving time. As a result, most computation is ideally amortized offline or nearline, with only a lean, predictable retrieval path online.
- **Challenge 3: Privacy Constraints** Enterprise agents must comply with strict data-governance policies (e.g., GDPR [7]). With these policies, the memory must enforce strict isolation, preventing cross-tenant retrieval, and support clean deletion when required.
- **Challenge 4: Adaptability** The memory framework must adapt continuously to shifting query patterns, rather than depending on brittle hand-crafted rules or feature-specific engineering pipelines.
- **Challenge 5: Observability** The memory system should preserve references to its underlying information sources and surface them during retrieval, enabling inspection, debugging, and governance. It should also remain consistent as the source of truth (SOT) evolves (e.g., edits, backfills, or deletions), preventing stale or contradictory memories.

To address these challenges, we propose HLTM, a hierarchical long-term semantic memory framework for industrial LLM agents. HLTM organizes memory into a tree-structured index that supports massively parallel execution and lossless incremental ingestion, generates multi-granularity memory representations for low-latency retrieval at serving time, and supports enhanced privacy policies by aligning the memory hierarchy with business scopes such as seats and projects. The framework is adaptable through automatic query pattern analysis and knowledge extraction. Each memory node

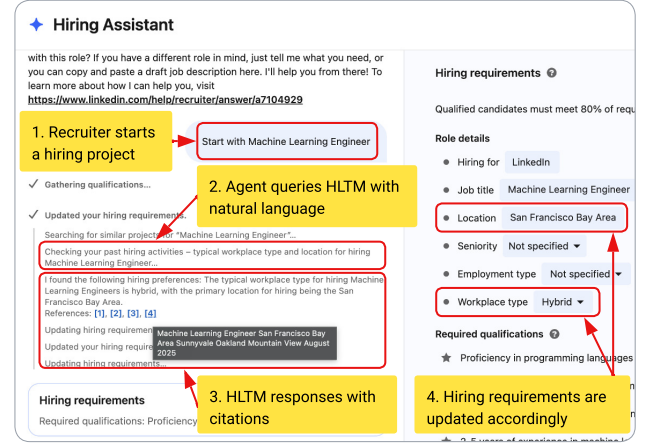


Figure 1: LinkedIn’s Hiring Assistant UI with HLTM: a recruiter initiates a hiring project; the Hiring Assistant queries HLTM in natural language to retrieve preference signals, then uses the returned information to update structured hiring requirements.

is bound to a stable business identifier, providing explicit provenance and observability for inspection, debugging, and governance. HLTM has been fully deployed in production as the long-term semantic memory of LinkedIn’s Hiring Assistant [8] (Figure 1). Over six months of real-world operation, it has demonstrated robust, reliable, enterprise-scale performance at controlled latency and cost (Figure 2).

Our main contributions are summarized below:

- **Production-scale long-term memory for enterprise agents:** We present an industrial-scale memory architecture that scales to millions of documents and supports lossless incremental updates. The system has been deployed in LinkedIn’s Hiring Assistant and serves production hiring workflows.
- **Hierarchical, tree-based memory representation:** We propose a tree-structured memory topology with multi-view memory representation, enabling massively parallel indexing, privacy-preserving retrieval scoping, and low-latency serving.
- **Adaptive ingestion via workload feedback:** We design a lightweight adaptation loop that mines historical query patterns to continuously refine memory content and improve retrieval quality and efficiency.

The remainder of the paper is organized as follows. Section 2 reviews related research and industrial practices in agent memory. Section 3 presents the design and methodology of HLTM. Section 4 provides a comparative evaluation against existing memory techniques. Section 5 describes HLTM’s production use case in LinkedIn’s Hiring Assistant. Finally, Section 6 concludes the paper and outlines directions for future work.

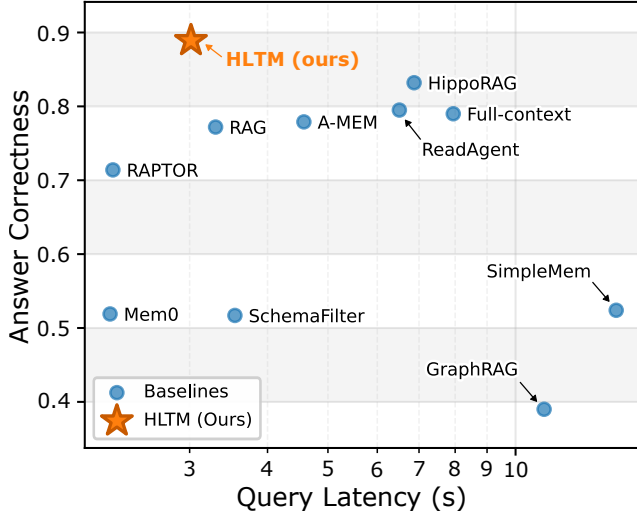


Figure 2: Performance–latency trade-off across evaluated methods on LinkedIn’s Hiring Assistant dataset. The HLTM consistently achieves higher answer correctness while maintaining lower latency compared to baseline approaches.

2 Related Work

Recent work has substantially improved agents’ long-context processing by introducing explicit memory mechanisms. MemGPT [17] and MemOS [14] adopt an OS-inspired design that swaps information between the limited context window and external memory. Mem0 [4] stores discrete memory items and retrieves them primarily via semantic search. MemoryBank [33] maintains timestamped interaction memories with time-based decay to emphasize recency and salience. ReadAgent [12] compresses long documents into compact gist memories with on-demand lookup of supporting passages. SCM [22] equips agents with a controller that adaptively decides when to store, update, or retrieve from a stream of past interactions.

Beyond unstructured memory stores, recent studies explore structured representations to improve organization and retrieval. GraphRAG [6] builds a knowledge graph and leverages community-based summarization to better capture entity-centric dependencies. HippoRAG [10] draws inspiration from hippocampal indexing, integrating graph-based retrieval with personalized PageRank for multi-hop reasoning and continual knowledge integration. A-Mem [26] uses Zettelkasten-style atomic notes with dynamic linking to support autonomous memory evolution, while G-Memory [30] generalizes this direction to multi-agent settings with hierarchical graph structures that model both reusable insights and collaboration trajectories. In parallel, RAPTOR [19] and TreeRAG [21] leverage hierarchical structures for long-context retrieval, combining multi-level organization with structure-aware access to improve downstream QA. MemTree [18] complements this line with an incrementally updated memory tree that maintains node-level summaries and embeddings as agent knowledge evolves.

Despite these advances, enterprise deployment remains challenging. Many methods lack principled memory partitioning aligned with access boundaries, raising privacy and scope-isolation risks.

Graph-based approaches often require extensive LLM usage during indexing and incur online latency, limiting scalability [32]. Tree-style methods frequently depend on the LLM’s world knowledge for aggregation, which can weaken domain-specific controllability [16]. Finally, most systems lack robust citation practices and provenance tracking, reducing transparency and debuggability in production [3].

3 Method

3.1 Overview

We propose HLTM, a hierarchical long-term semantic memory that organizes information in a tree: higher-level nodes capture broad patterns, while lower-level nodes store fine-grained, entity-specific information. Together, these levels support both generic and highly targeted queries within a unified structure (Figure 3). In production, the tree topology aligns with business scope and access-control boundaries, enabling privacy enforcement by restricting retrieval to appropriate subtrees (Section 3.2). Node contents are built offline via data-driven preference extraction and hierarchical aggregation (Sections 3.3–3.4), and are queried at serving time using hybrid embedding-based retrieval and answered via in-context learning (Section 3.5). HLTM also supports adaptation to evolving query patterns and *lossless* incremental updates, enabling nearline freshness at low indexing cost (Sections 3.6–3.7).

3.2 Schema-aligned Hierarchical Topology

Most hierarchical memory systems adopt *semantic hierarchies*, clustering embeddings of text segments into progressively coarser nodes [19, 21]. While effective for organizing free-form text, this approach is ill-suited to multi-tenant enterprise privacy requirements: clustering can mix content across business scopes, creating privacy and compliance risks under strict isolation and access-control policies [9, 29]. Semantic hierarchies are also brittle under incremental updates: new data often triggers re-clustering and topology changes, while keeping the topology fixed can lead to quality drift and periodic full re-indexing.

Hierarchical schemas are ubiquitous in enterprise data (ownership chains), documents (structural nesting), and conversations (sessions and messages). HLTM exploits this by constructing a *schema-aligned hierarchy* whose topology is dictated by the enterprise data model and ownership boundaries. In the hiring domain, for example, leaf nodes represent fine-grained work units (e.g., hiring projects), intermediate nodes represent recruiter seats, and root nodes represent organizational accounts. We formalize this as a tree index $\mathcal{T} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ contains business entities and edges $\mathcal{E}$ follow schema-defined parent–child relations. Each node $v \in \mathcal{V}$ owns the memory for its entity, and retrieval is restricted to subtrees consistent with the query scope and access-control policy.

This schema-aligned design provides three deployment-critical benefits: (i) **privacy-aware scoping** via subtree-restricted retrieval, enabling per-entity isolation and controlled sharing; (ii) **localized incremental updates** that touch only affected leaves and their ancestor nodes, reducing update cost from $O(|\mathcal{V}|)$ to the tree height; and (iii) **topology stability** over time, avoiding drift and periodic full re-indexing commonly seen in clustering-based hierarchies.

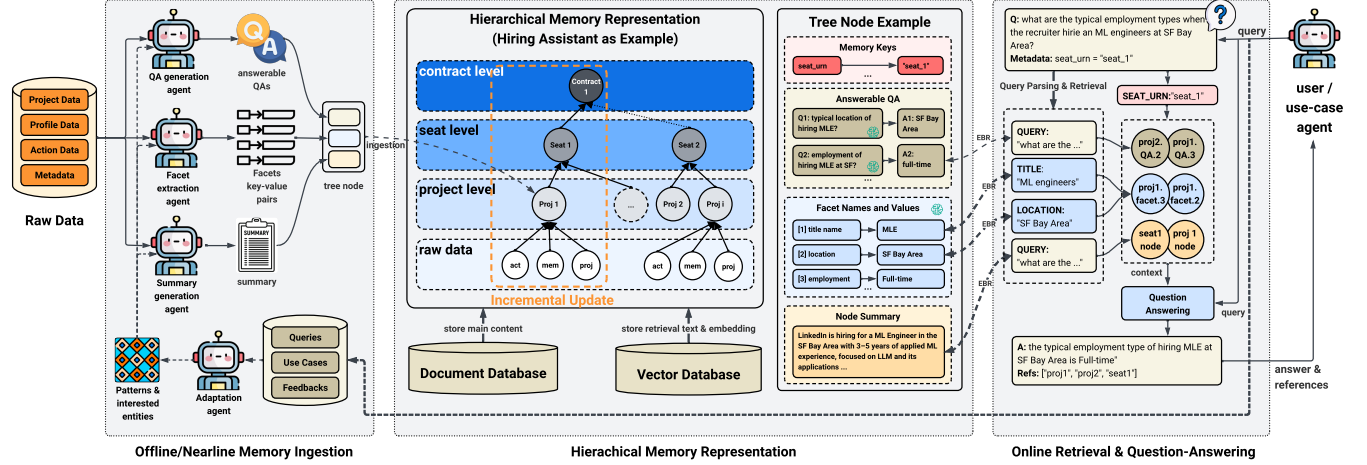


Figure 3: Overview of HLTM. Ingestion agents convert raw data into multi-view representations (left) and organize them in a schema-aligned memory tree (mid). At inference, relevant nodes are retrieved to generate answers (right).

3.3 Memory Representation

Rather than storing raw data at each node, we transform node content D_v for node $v \in \mathcal{V}$ into a compact, multi-view memory representation M_v :

$$M_v = (F_v, Q_v, S_v)$$

where F_v , Q_v , and S_v denote the facet, QA, and summary views at node $v \in \mathcal{V}$, respectively. The full prompt templates for the three agents are provided in Appendix A.

- **Facet (Key-Value) Representation.** Key-value pairs provide a structured and interpretable view of salient facts, making them easy to index, filter, and retrieve. We prompt an LLM to extract a set of facet pairs $F_v = \{(key, value)\}$ from D_v and store them in a document database. To support semantic retrieval, we linearize each pair (e.g., location: SF Bay Area) and compute an embedding for indexing.
- **Answerable-QA Representation.** Question-answer pairs align with how users naturally phrase queries, so embedded questions often lie closer to user queries in embedding space, improving retrieval accuracy. They also enable a “think-fast” serving mode by precomputing answerable questions offline, minimizing online computation. Concretely, we prompt an LLM to generate answerable QA pairs $Q_v = \{(q, a)\}$ from the raw data D_v and store them in a document database. To support semantic retrieval, we embed and index each question in a vector store.
- **Summary Representation.** Some knowledge is inherently unstructured (e.g., nuanced narratives or multi-step logic) and is not naturally captured by key-value or QA formats; therefore, we use natural-language summarization as the default representation. Specifically, we prompt an LLM to first compress D_v into a detailed paragraph S_v that preserves key information, and then distill it into a single-sentence summary to compute embeddings for vector-store indexing.

3.4 Hierarchical Memory Aggregation

Summarization-style queries that ask for a summary or synthesis over a broad scope typically require aggregating evidence from multiple leaf nodes [6]. A straightforward strategy is to retrieve a large set of relevant leaf-level documents at query time and ask the LLM to summarize them on the fly; however, this approach is often impractical for three reasons: (i) **high online cost**, because summarizing detail-heavy evidence is both token-intensive and latency-critical; (ii) the **needle-in-a-haystack effect**, where relevant signals are drowned out by irrelevant details, increasing the likelihood of omissions and factual drift; and (iii) **embedding-space misalignment**, where broad summarization queries may not align well with leaf-level embeddings, leading to poor recall and unstable retrieval quality.

To address these challenges, HLTM performs *offline hierarchical aggregation*, composing child memories into broader-scope parent representations in a bottom-up manner. Aggregation follows the parent-child relations defined by the tree topology (Section 3.2) and is applied recursively up to the root. For each parent node $p \in \mathcal{V}$, we collect its children’s memory representations $\{M_c\}$ as input and prompt an LLM agent to generate parent-level representations in the same multi-view form. Specifically, we guide the LLM to (i) group semantically similar evidence across children, (ii) merge redundant content and reconcile conflicts, and (iii) optionally prune low-salience details that rarely recur across children.

3.5 Memory Retrieval and Answer Generation

HLTM performs *identity-scoped multi-signal retrieval* to efficiently retrieve privacy-compliant memory content for a given user query. Building on RAPTOR’s collapsed-tree paradigm [19], which collapses nodes across levels into a single candidate pool for retrieval, we introduce two production-critical extensions: (i) **identity-scoped subtree filtering** to enforce enterprise access boundaries, and (ii) **multi-signal retrieval** to robustly surface relevant memories under diverse query formulations.

3.5.1 Identity-scoped Subtree Filtering. Each production query is issued under an explicit identity scope that maps to a node v in our schema-aligned hierarchy (Section 3.2). We enforce this scope via *hard filtering*: retrieval candidates are restricted to the subtree $\mathcal{T}_v$ rooted at node v , guaranteeing strict tenant isolation. For example, a recruiter-scoped query may access only the recruiter node and its descendants; nodes owned by other recruiters are excluded. Formally,

$$\mathcal{T}_v = \mathcal{T}[\{v\} \cup \text{Desc}(v)]$$

3.5.2 Multi-signal Retrieval. Within the privacy-scoped subtree $\mathcal{T}_v$, we build on RAPTOR’s collapsed-tree retrieval method [19] and extend it with multi-retriever scoring. Rather than relying on a single similarity signal, we rank candidates using three complementary retrievers tied to the node representations in Section 3.3: the facet-based retriever capturing structured constraints, the answerable-QA retriever capturing query-like intents, and the summary-based retriever capturing free-form semantics. Each retriever produces a relevance score, and we return the top- k memories per view as retrieval context.

- **Facet-based Retrieval:** We enable fine-grained, schema-aware matching by parsing the query q into a set of facet key-value pairs F_q . Each pair is then linearized into a micro-query (“key: val”) and compared against the offline-constructed facet strings in F_v for each node v in embedding space. For example, the query “typical workplace for hiring software engineers in the San Francisco Bay Area?” is decomposed into two micro-queries: “title: software engineer” and “location: San Francisco Bay Area”. We aggregate similarities into a query-node score $s_{\text{facet}}(q, v)$ by averaging over the top- k most similar node facets per micro-query, preventing irrelevant facet pairs from diluting the score, and return the facet sets $F_k(q)$ from the top- k_{facet} nodes.

$$s_{\text{facet}}(q, v) = \frac{1}{k |F_q|} \sum_{f \in F_q} \sum_{f' \in \text{TopK}(f, F_v)} \cos(\text{emb}(f), \text{emb}(f'))$$

- **Answerable-QA Retrieval:** We match the query q against a node’s offline-generated answerable question set Q_v . Specifically, we embed q and each $q' \in Q_v$, compute cosine similarities, and score node v by the best-matching question $s_{\text{QA}}(q, v)$. We then return the top- k_{QA} answerable question-answer pairs $Q_k(q)$.

$$s_{\text{QA}}(q, v) = \max_{q' \in Q_v} \cos(\text{emb}(q), \text{emb}(q'))$$

- **Summary-based Retrieval:** We embed the query q and each node’s offline-generated summary view S_v , rank nodes by cosine similarity, and return the top- k_{summary} summaries $S_k(q)$.

$$s_{\text{summary}}(q, v) = \cos(\text{emb}(q), \text{emb}(S_v))$$

3.5.3 Answer Generation with Retrieved Context. After retrieval, we apply a standard in-context learning prompt that conditions the LLM on the top-ranked memory contents as context. In addition to the final answer, the system also returns *references*, namely the IDs of the retrieved memory tree nodes used as grounding evidence (Figure 1), enabling provenance tracking and downstream inspection. The prompt template is provided in Appendix A.4.

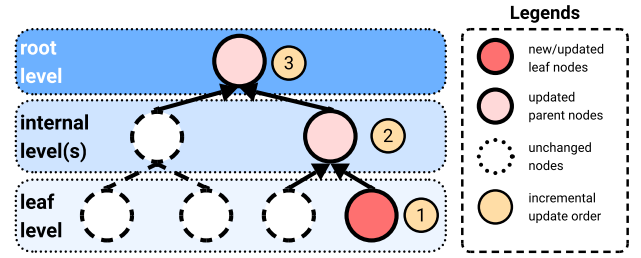


Figure 4: Lossless incremental nearline indexing in HLTM: updated leaves are re-indexed and their aggregates propagated along ancestor paths, while unaffected subtrees remain unchanged.

$$\text{answer, references} = \text{LLM}(q, F_k(q) \cup Q_k(q) \cup S_k(q))$$

3.6 Adaptation via Query Pattern Analysis

To remain adaptive under continuously shifting production query patterns and an evolving document corpus, we introduce a periodic adaptation step that distills historical query workloads into signals for memory construction, rather than relying solely on an LLM’s world knowledge to determine what is important to memorize. Over a sliding window of past queries, the module extracts (i) frequently recurring query patterns and (ii) salient facet names. The query patterns act as priors for the Answerable-QA Generation Agent, guiding it to synthesize answerable questions from node raw data D_v , while facet names provide optional hints for the Facet Extraction Agent and Summary Agents to emphasize coverage of high-impact attributes during memory construction and tree aggregation. To reduce overfitting, we apply minimum-support thresholds (i.e., only patterns/facets supported by multiple queries are retained); for sensitive production settings, we optionally require human review before deploying updates to the indexing pipeline.

3.7 Lossless Incremental Nearline Indexing

Data freshness is critical for user-facing agents; for example, recruiter memories in LinkedIn’s Hiring Assistant should reflect new interactions within minutes. Full indexing that re-runs all LLM-based summarization and embedding steps is too expensive; incremental updates such as GraphRAG [6] are typically *lossy* and still require periodic full re-indexing.

HLTMs natively supports lossless incremental nearline indexing, keeping large-scale memories fresh without full re-indexing. As shown in Figure 4, we maintain a set V^* of leaf nodes newly created or modified since the last cycle; for each $v \in V^*$, we recompute its memory representation M_v as discussed in Section 3.3, and then update only its ancestor path $\{u \in V \mid v \preceq u\}$ up to the root. Because each parent is always recomputed from its latest children, the resulting tree is identical to a full rebuild over the latest data snapshot. This enables nearline update windows of a few minutes that are substantially cheaper than full re-indexing while preserving the semantics of a fresh global rebuild. We evaluated the gain from incremental indexing in production in Appendix D.

4 Experiments

4.1 Dataset

LinkedIn’s Hiring Assistant Dataset. We constructed a human-labeled benchmark dataset derived from LinkedIn’s Hiring Assistant usage logs. The dataset spans diverse hiring domains, with gold reference answers curated by domain experts. Each data point contains (i) **raw hiring documents**, including a recruiter’s historical hiring projects, within-project interactions on candidates, and anonymized candidate and project data; (ii) **memory queries** issued by the Hiring Assistant, spanning both *summary-style* queries that aggregate signals across projects to infer preferences (e.g., typical titles, locations, and qualifications) and *retrieval-style* queries that request targeted projects or evidence snippets for downstream reasoning; and (iii) **gold answers**, which are personalized to the underlying hiring documents. To improve label reliability, each answer is independently annotated by at least three annotators, and disagreements are resolved via majority voting.

LongMemEval-s Dataset. To assess HLTm’s performance beyond the hiring domain, we additionally evaluate on LongMemEval-s [25], a publicly available benchmark designed to stress-test long-term memory in conversational AI systems.

Table 1: Dataset summary statistics.

Dataset Metrics	LinkedIn Dataset	LongMemEval-s
# Queries	1,341	500
– Retrieval queries	473	—
– Summarization queries	868	—
Avg. history length (tokens)	≈40K	≈103K
Total history length (tokens)	≈53M	≈51M

4.2 Evaluation Metrics

4.2.1 Performance Metrics.

Lexical Similarity Metrics. Prior work in conversational AI and agent memory evaluation [4, 15, 26] commonly reports lexical overlap metrics. We report token-level F1 to balance spurious content and missing coverage, and BLEU-1 to provide a complementary, precision-oriented lexical similarity signal.

Semantic Correctness Metrics. To evaluate answer correctness beyond surface-form overlap, we additionally employ an LLM-as-a-judge score. The judge is given the query, the system-generated answer, and the gold reference answer, and scores factual consistency with the reference by checking key constraints such as numbers and named entities. The judge is instructed to rely only on the gold reference and outputs a scalar score in $[0, 1]$ along with a brief rationale. To validate reliability, we conducted a human alignment study to ensure the LLM judge agrees with expert judgment. Specifically, we collected approximately 200 query-answer triples independently annotated by 2–4 domain experts; inter-annotator agreement (Quadratic Weighted Cohen’s κ) exceeded 0.8, and the LLM judge benchmarked against majority-vote consensus labels achieved the same threshold, confirming strong alignment with human judgment.

4.2.2 Deployment Metrics.

Latency. We measure both *offline indexing latency* and *online query latency*. Offline indexing latency indicates whether a method can refresh and maintain memory at production scale (e.g., millions of documents ingested daily for LinkedIn’s Hiring Assistant). Online query latency directly affects end-user experience in interactive, chat-based workflows. When a method supports parallelism, we run indexing and retrieval with a fixed concurrency setting to reflect realistic deployment conditions.

LLM Invocation Counts and Token Consumption. We track both LLM call counts and token usage, since together they largely determine serving cost. Our accounting covers all LLM interactions during the query phase, including query rewriting, planning, retrieval-time processing, and final answer generation. We compute token usage with a consistent tokenizer (o200k_base).

4.3 Baselines

To comprehensively evaluate HLTm, we compare against both basic baselines and advanced RAG/memory methods. We hold the backbone LLM and embedding model fixed across methods, and evaluate all systems under the same serving setup and context-window constraints.

4.3.1 Basic Baselines. We include three simple yet strong baselines, following the evaluation practice of Mem0 [4], to isolate the value of an external memory module. (i) **Conventional retrieval-augmented generation (RAG)** [13] chunks documents into fixed-length segments, embeds each chunk, and retrieves the top chunks by semantic similarity as context for question answering. (ii) **Full-context prompting** feeds the full available history (or the maximal portion that fits within the LLM context window) directly into the LLM, providing an architecture-free baseline. (iii) **Schema filter** parses each document into a structured table at ingestion time and applies query parsing at retrieval time to filter matching documents, which are then passed to the LLM for answer synthesis. This baseline directly exploits domain schema knowledge, isolating the contribution of HLTm’s hierarchical aggregation and multi-view memory.

4.3.2 Advanced Baselines. We further compare against representative advanced RAG and memory systems that introduce explicit structure beyond flat retrieval. (i) **Graph-based methods** represent memory as nodes and edges, enabling entity-centric organization and multi-hop retrieval; we include GraphRAG [6], HippoRAG [10], A-Mem [26], and SimpleMem [15]. (ii) **Tree-based methods** build hierarchical indices via recursive clustering and summarization to balance coverage and retrieval efficiency; we evaluate RAPTOR [19], and leave TreeRAG [21] and MemTree [18] for future comparison once runnable implementations are publicly available. (iii) **Other common baselines** include ReadAgent [12] and Mem0 [4], which serve as widely used reference points for long-context retrieval and memory-augmented generation.

4.4 Implementation Details

HLTM was originally deployed in LinkedIn’s production environment with PySpark-based offline ingestion and in-house document/vector stores. For this study, we re-implemented HLTM as a standalone Python library for benchmarking. All methods run on Python 3.12 and access LLMs via the Azure OpenAI Service API. We use text-embedding-3-large for embeddings, GPT-4o mini as the primary LLM, and four widely-used open-source language models ranging from 27B to 106B parameters. We report GPT-4o mini results in the main paper and provide open-source models’ results in Appendix B.1. Baselines use default open-source settings unless noted, and all results are averaged over three runs.

4.5 Experiment Result Analysis

4.5.1 Overview. Figure 2 shows that HLTM achieves the best quality-latency trade-off on our benchmark, delivering the highest correctness (0.892) at low query latency (≈ 3 s). Methods with similar latency (e.g., RAG) are substantially less accurate, whereas higher-performing baselines such as HippoRAG [10] achieve improved correctness only at a substantially higher latency (≈ 7 s).

4.5.2 Quality Analysis. Table 2 reports results on LinkedIn’s Hiring Assistant dataset using GPT-4o mini. HLTM consistently outperforms all ten baselines across both query types¹. For summary-style queries, HLTM improves semantic correctness by more than 5% over the strongest baseline, HippoRAG (0.833). For retrieval-style queries, HLTM improves retrieval F1 by more than 10% over the best baseline, ReadAgent (0.617), demonstrating a favorable precision-recall trade-off. Both gains are statistically significant under a Wilcoxon signed-rank test ($p < 10^{-7}$), with Cliff’s $\delta = +0.058$ for summary-style queries against HippoRAG and $\delta = +0.328$ for retrieval-style queries against ReadAgent. Table 5 in Appendix B.1 further shows that these improvements are consistent across four open-source language models ranging from 27B to 106B parameters. Table 3 reports results on LongMemEval-s, which probes long-term memory across six question types spanning knowledge updates, multi-session reasoning, single-session retrieval (assistant, user-preference, and user-information views), and temporal reasoning. HLTM again outperforms all baselines, improving overall correctness by 6% over the strongest baseline, A-Mem (0.716), and achieves the best score on every one of the six question types. One of the largest gains appears on multi-session reasoning (0.632 vs. A-Mem 0.556), which most directly stresses cross-session aggregation that HLTM’s hierarchical aggregation is designed to handle.

4.5.3 Latency and Cost Analysis. Figure 5 highlights the trade-off between offline indexing latency and online query latency. HLTM pushes the Pareto frontier by enabling parallel indexing across child nodes and reducing online latency through collapsed-tree retrieval. Token analysis further shows that HLTM reduces query-time token usage by at least 50% relative to graph-based baselines, indicating more efficient context selection and compaction (Table 6 in Appendix B.2).

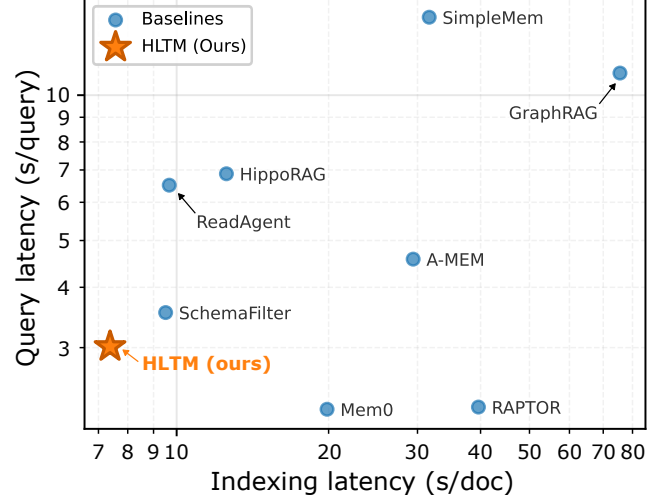


Figure 5: Query vs. indexing latency: HLTM advances the Pareto frontier.

4.6 Ablation Study and Analysis

We conduct an ablation study to quantify the contributions of tree aggregation, adaptation, and each memory representation. Table 4 shows that disabling **tree aggregation** (i.e., restricting retrieval to leaf nodes only) causes a clear drop in summary-style correctness (0.892→0.850; ≈ 4 points) and a large drop in retrieval-style F1 (0.782→0.618; ≈ 16 points). Removing **adaptation** also degrades quality (correctness 0.892→0.829; ≈ 6 points; retrieval F1 0.782→0.593; ≈ 19 points), indicating that aligning aggregation with workload-specific query patterns remains important even when the core hierarchy is intact. Finally, single-representation variants suggest that **summary** memory contributes most to overall performance, while **facet** and **QA** memories provide complementary gains; combining all three yields the best correctness on summary-style queries and the best F1 on retrieval-style queries.

4.7 Privacy Discussion

Privacy is a first-class requirement for production memory systems. In HLTM, isolation is enforced structurally: each node is bound to an identity scope (e.g., recruiter, hiring project), and retrieval is restricted to the subtree rooted at that scope. Recruiter-scoped queries can access only that recruiter node and its owned projects; project-scoped queries are further constrained to a single project. This business-aligned hierarchy lets us *index once* and *query at multiple levels* without per-level re-indexing or global LLM routing, while preserving strict privacy isolation (Table 7 in Appendix C).

5 Production Use Case

LinkedIn’s Hiring Assistant [8] is an AI recruiting agent that helps recruiters discover, evaluate, and engage candidates with greater speed and scale. Architecturally, LinkedIn’s Hiring Assistant is a plan-and-execute system centered on a supervisor agent that interprets recruiter intent and orchestrates specialized sub-agents and tools (Figure 6). HLTM has been fully ramped as the long-term semantic memory of LinkedIn’s Hiring Assistant since early

¹Disclaimer: Results may vary in production environments or with different datasets.

Table 2: LinkedIn dataset answer performance; Values are mean $\pm$ SEM; $\uparrow$ indicates higher is better.

Methods	Summary-style Queries			Retrieval-style Queries		
	Token-F1 $\uparrow$	BLEU-1 $\uparrow$	Correctness $\uparrow$	Precision $\uparrow$	Recall $\uparrow$	F1 $\uparrow$
FULL-CONTEXT	0.478 $\pm$ 0.007	0.348 $\pm$ 0.007	0.791 $\pm$ 0.009	0.465 $\pm$ 0.017	0.849 $\pm$ 0.015	0.544 $\pm$ 0.015
RAG	0.534 $\pm$ 0.008	0.409 $\pm$ 0.008	0.770 $\pm$ 0.011	0.557 $\pm$ 0.017	0.762 $\pm$ 0.018	0.604 $\pm$ 0.016
SCHEMA FILTER	0.352 $\pm$ 0.011	0.269 $\pm$ 0.010	0.513 $\pm$ 0.016	0.541 $\pm$ 0.022	0.575 $\pm$ 0.022	0.546 $\pm$ 0.021
A-MEM	0.523 $\pm$ 0.007	0.391 $\pm$ 0.007	0.779 $\pm$ 0.010	0.498 $\pm$ 0.017	0.719 $\pm$ 0.019	0.549 $\pm$ 0.017
HIPPO-RAG	0.453 $\pm$ 0.005	0.329 $\pm$ 0.005	0.833 $\pm$ 0.010	0.515 $\pm$ 0.018	0.718 $\pm$ 0.019	0.560 $\pm$ 0.017
RAPTOR	0.539 $\pm$ 0.008	0.423 $\pm$ 0.008	0.715 $\pm$ 0.012	0.332 $\pm$ 0.018	0.425 $\pm$ 0.021	0.348 $\pm$ 0.018
GRAPH-RAG	0.114 $\pm$ 0.003	0.074 $\pm$ 0.002	0.381 $\pm$ 0.013	0.082 $\pm$ 0.011	0.112 $\pm$ 0.014	0.084 $\pm$ 0.011
SIMPLE-MEM	0.150 $\pm$ 0.006	0.032 $\pm$ 0.003	0.518 $\pm$ 0.015	0.445 $\pm$ 0.019	0.570 $\pm$ 0.022	0.470 $\pm$ 0.020
MEM0	0.470 $\pm$ 0.008	0.372 $\pm$ 0.008	0.521 $\pm$ 0.013	0.376 $\pm$ 0.020	0.412 $\pm$ 0.022	0.369 $\pm$ 0.020
READ-AGENT	0.473 $\pm$ 0.007	0.347 $\pm$ 0.008	0.796 $\pm$ 0.009	0.542 $\pm$ 0.016	0.861 $\pm$ 0.014	0.617 $\pm$ 0.015
HLTM (ours)	0.724 $\pm$ 0.008	0.588 $\pm$ 0.008	0.892 $\pm$ 0.007	0.761 $\pm$ 0.016	0.874 $\pm$ 0.014	0.782 $\pm$ 0.014

Table 3: LongMemEval-s answer accuracy, overall and per query type; Values are mean $\pm$ SEM.

Methods	Overall	By Question Type					
		know-update	multi-sess	single-sess-asst	single-sess-pref	single-sess-user	temp-reason
FULL-CONTEXT	0.494 $\pm$ 0.022	0.679 $\pm$ 0.053	0.406 $\pm$ 0.043	0.607 $\pm$ 0.065	0.067 $\pm$ 0.046	0.814 $\pm$ 0.046	0.353 $\pm$ 0.041
RAG	0.435 $\pm$ 0.017	0.415 $\pm$ 0.034	0.207 $\pm$ 0.024	0.923 $\pm$ 0.022	0.528 $\pm$ 0.069	0.783 $\pm$ 0.031	0.264 $\pm$ 0.027
A-MEM	0.716 $\pm$ 0.020	0.846 $\pm$ 0.040	0.556 $\pm$ 0.043	0.946 $\pm$ 0.030	0.567 $\pm$ 0.090	0.886 $\pm$ 0.038	0.647 $\pm$ 0.041
HIPPO-RAG	0.578 $\pm$ 0.022	0.667 $\pm$ 0.053	0.534 $\pm$ 0.043	0.946 $\pm$ 0.030	0.100 $\pm$ 0.054	0.829 $\pm$ 0.045	0.391 $\pm$ 0.042
RAPTOR	0.530 $\pm$ 0.022	0.577 $\pm$ 0.056	0.376 $\pm$ 0.042	0.821 $\pm$ 0.051	0.433 $\pm$ 0.091	0.871 $\pm$ 0.040	0.376 $\pm$ 0.042
MEM0	0.462 $\pm$ 0.023	0.564 $\pm$ 0.057	0.444 $\pm$ 0.043	0.071 $\pm$ 0.034	0.400 $\pm$ 0.089	0.871 $\pm$ 0.040	0.383 $\pm$ 0.042
READ-AGENT	0.480 $\pm$ 0.023	0.538 $\pm$ 0.057	0.316 $\pm$ 0.040	0.821 $\pm$ 0.051	0.467 $\pm$ 0.091	0.771 $\pm$ 0.050	0.316 $\pm$ 0.040
HLTM (ours)	0.778 $\pm$ 0.019	0.872 $\pm$ 0.037	0.632 $\pm$ 0.042	0.964 $\pm$ 0.025	0.667 $\pm$ 0.086	0.986 $\pm$ 0.014	0.707 $\pm$ 0.039

Table 4: HLTm ablation results on LinkedIn dataset answer performance; Values are mean $\pm$ SEM; $\uparrow$ indicates higher is better.

Methods	Summary-style Queries			Retrieval-style Queries		
	Token-F1 $\uparrow$	BLEU-1 $\uparrow$	Correctness $\uparrow$	Precision $\uparrow$	Recall $\uparrow$	F1 $\uparrow$
HLTM	0.724 $\pm$ 0.008	0.588 $\pm$ 0.008	0.892 $\pm$ 0.008	0.761 $\pm$ 0.016	0.874 $\pm$ 0.014	0.782 $\pm$ 0.014
HLTM (w/o aggregation)	0.672 $\pm$ 0.007	0.525 $\pm$ 0.007	0.850 $\pm$ 0.008	0.568 $\pm$ 0.018	0.784 $\pm$ 0.017	0.618 $\pm$ 0.016
HLTM (w/o adaptation)	0.689 $\pm$ 0.008	0.555 $\pm$ 0.009	0.829 $\pm$ 0.010	0.524 $\pm$ 0.016	0.811 $\pm$ 0.016	0.593 $\pm$ 0.015
HLTM (facet-only)	0.619 $\pm$ 0.010	0.502 $\pm$ 0.009	0.809 $\pm$ 0.011	0.552 $\pm$ 0.018	0.755 $\pm$ 0.019	0.599 $\pm$ 0.017
HLTM (QA-only)	0.661 $\pm$ 0.009	0.547 $\pm$ 0.010	0.710 $\pm$ 0.012	0.642 $\pm$ 0.019	0.717 $\pm$ 0.019	0.646 $\pm$ 0.018
HLTM (summary-only)	0.680 $\pm$ 0.008	0.537 $\pm$ 0.008	0.850 $\pm$ 0.008	0.684 $\pm$ 0.017	0.877 $\pm$ 0.014	0.730 $\pm$ 0.015

2026 (Figure 1). Concretely, HLTm participates in the user-agent chat process by dynamically recalling each recruiter’s long-term preferences from historical projects and activities, enabling the assistant to personalize interactions without requiring recruiters to re-specify requirements that can be inferred from past behavior.²

We assess HLTm’s online impact by analyzing user satisfaction across 3,000+ eligible production conversations from the initial roll-out. The supervisor agent autonomously decides when to invoke the memory module based on conversational context, triggering it

in over 40% of sessions — a strong signal that a large fraction of recruiter interactions benefit from long-term preference recall. Across 1,000+ recruiter seats, sessions in which HLTm was invoked show a **5–10 percentage-point reduction in negative feedback rate** during the hiring-calibration phase, statistically significant under a two-proportion z-test ($p < 0.01$). This improvement indicates that candidates and recommendations surfaced during calibration are of noticeably higher quality when the assistant leverages historical preference memory, demonstrating HLTm’s tangible impact on real recruiter workflows at production scale.

²Disclaimer: All memory data is scoped to the recruiter’s environment and is not used for training content-generating AI models; customers retain full control over their stored memory with robust privacy and management options.

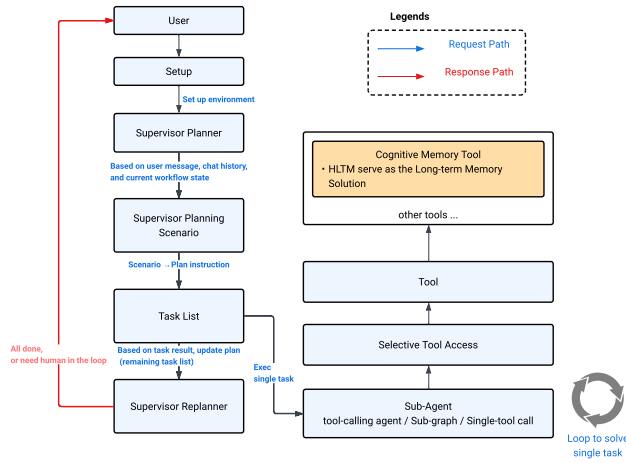


Figure 6: HLTM's Production Use Case in LinkedIn's Hiring Assistant.

6 Conclusion

We introduce HLTM, a hierarchical, schema-aligned long-term memory framework for industrial LLM agents, and demonstrate its end-to-end deployment in LinkedIn's Hiring Assistant [8]. HLTM organizes memories into a tree topology that mirrors real-world business scopes (Section 3.2) and generates memory content via complementary multi-view representations with bottom-up hierarchical aggregation (Sections 3.3–3.4). Experiments on production hiring data show that HLTM consistently outperforms strong RAG and memory baselines, with semantic correctness gains of more than 5% on summary-style queries and F1 gains of more than 10% on retrieval-style queries³, while maintaining favorable latency and token cost for real-time, chat-based workflows (Section 4.5). In production, HLTM supports large-scale indexing with freshness guarantees via lossless incremental ingestion (Section 3.7) and provides privacy and observability through business-keyed, transparent nodes (Section 4.7), making it a practical foundation for industrial GenAI products. HLTM has been fully deployed in LinkedIn's Hiring Assistant to support real recruiter hiring workflows (Section 5). Looking ahead, we plan to extend HLTM to richer cross-entity reasoning, multimodal signals, and additional agent use cases beyond hiring, and to explore tighter integration with agent planners and controllers for end-to-end adaptive systems.

Acknowledgments

This work was conducted by the Agent Platform team at LinkedIn. We thank our collaborators on the LinkedIn Talent Solutions team for building the Hiring Assistant product and for partnering with us to integrate HLTM as its long-term memory solution. We also thank our leadership team, especially Anuradha Krishnan, Deepak Agarwal, Balaji Krishnapuram, and Erran Berger for their guidance and support.

³Disclaimer: Results may vary in production environments or with different datasets.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Anthropic. 2025. Home — Anthropic. <https://www.anthropic.com/>. Accessed: 2025-12-12.
- [3] Sizhe Cheng, Jiaping Li, Huanchen Wang, and Yuxin Ma. 2025. Ragtrace: Understanding and refining retrieval-generation dynamics in retrieval-augmented generation. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–20.
- [4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. In *Proceedings of the 28th European Conference on Artificial Intelligence*. 2993–3000. doi:10.3233/FAIA251160
- [5] Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multi-modality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [6] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [7] GDPR.eu. [n. d.]. Complete guide to GDPR compliance. <https://gdpr.eu/>. Accessed: 2026-01-30.
- [8] Xiaoyang Gu, Xie Lu, and Daniel Hewlett. 2025. *Building the agentic future of recruiting: how we engineered LinkedIn's Hiring Assistant*. LinkedIn Engineering. <https://www.linkedin.com/blog/engineering/ai/how-we-engineered-linkedin-hiring-assistant>
- [9] Changyue Jiang, Xudong Pan, Geng Hong, Chenfu Bao, and Min Yang. 2024. Rag-thief: Scalable extraction of private data from retrieval-augmented generation applications with agent-based attacks. *arXiv preprint arXiv:2411.14110* (2024).
- [10] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 59532–59569.
- [11] Ehsan Kamalloo, Nouha Dziri, Charles Clarke, and Davood Rafiei. 2023. Evaluating open-domain question answering in the era of large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 5591–5606.
- [12] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. 2024. A human-inspired reading agent with gist memory of very long contexts. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 26396–26415.
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [14] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. 2025. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724* (2025).
- [15] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026. SimpleMem: Efficient Lifelong Memory for LLM Agents. *arXiv preprint arXiv:2601.02553* (2026).
- [16] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 1906–1919. doi:10.18653/v1/2020.acl-main.173
- [17] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. (2023).
- [18] Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2024. From isolated conversations to hierarchical schemas: Dynamic tree memory representation for llms. *arXiv preprint arXiv:2410.14052* (2024).
- [19] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.
- [20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [21] Wenyu Tao, Xiaofen Xing, Yirong Chen, Linyi Huang, and Xiangmin Xu. 2025. Treerag: Unleashing the power of hierarchical storage for enhanced knowledge retrieval in long documents. In *Findings of the Association for Computational Linguistics: ACL 2025*. 356–371.

- [22] Bing Wang, Xinnian Liang, Jian Yang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun Li. 2025. SCM: Enhancing large language model with self-controlled memory framework. In *Proceedings of the 30th International Conference on Database Systems for Advanced Applications*.
- [23] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [24] Jason Wei, Xuezhong Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [25] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*.
- [26] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*.
- [27] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [28] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2023. Large language models meet nl2code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7443–7464.
- [29] Shenglai Zeng, Jiankun Zhang, Pengfei He, Yiding Liu, Yue Xing, Han Xu, Jie Ren, Yi Chang, Shuaiqiang Wang, Dawei Yin, et al. 2024. The good and the bad: Exploring privacy issues in retrieval-augmented generation (rag). In *Findings of the Association for Computational Linguistics: ACL 2024*. 4505–4524.
- [30] Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. 2025. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. *arXiv preprint arXiv:2506.07398* (2025).
- [31] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.
- [32] Yibo Zhao, Jiapeng Zhu, Ye Guo, Kangkang He, and Xiang Li. 2025. E²GraphRAG: Streamlining Graph-based RAG for High Efficiency and Effectiveness. *arXiv preprint arXiv:2505.24226* (2025).
- [33] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731.
- [34] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2025. Large language models for information retrieval: A survey. *ACM Transactions on Information Systems* 44, 1 (2025), 1–54.

A HLTm Indexing-time Prompts

A.1 Facet Extraction

```
<system message>
  You are a helpful assistant specializing in extracting
  structured information from the input text data. Only
  retain key information; no need to include all details (e
  .g., detailed activities).
  Return a JSON object with key "facets\" and value as a
  flattened dictionary, in which the key is the facet name
  and the value is the corresponding extracted facet value
  in string format; no nested information.

  {{
    "facets": {{
      <facet_name>: <facet_value>,
      <facet_name>: <facet_value>,
      <facet_name>: <facet_value>,
      ...
    }}
  }}
</system message>

<user message> {document} </user message>
```

A.2 Question–Answer Generation

```
<system message>
  Analyze text, produce 5-10 self-contained Q&A pairs covering
  only the most critical info, and include a rationale
  explaining the result.

  Rules
  - Q&A must be explicit (no pronouns/ambiguous refs), concise,
    and essential-only.
  Output (single JSON)
  {{
    "rationale": "...",
    "question_answers": [
      {{ "question": "...", "answer": "...", "source": "<doc_id
      >" }}],
    ...
  }}
</system message>

<user message>
  {document}
</user message>
```

A.3 Summary Generation

A.3.1 Detailed Summary Generation.

```
<system message>
  You are a helpful assistant. Summarize this input document to
  retain key information.
  Please include the document ID(s) in the output when provided.
</system message>

<user message>
  {document}
</user message>
```

A.3.2 Concise Summary Generation.

```
<system message>
  You are a helpful assistant. You are provided with the
  following data, please generate a single-sentence summary
  of this data, only retain key information.
</system message>

<user message>
  {detailed_summary}
</user message>
```

A.4 Question Answering

```
<system message>
  You are a helpful assistant that generates an answer from a
  provided context.
  Output Format: a JSON object of this schema:
  {{
    "rationale": str, # rationale of generating the answer,
    "answer": str, # the answer to the query
    "citation": list[str], # a ranked list of node IDs provided
    in the context.
  }}
</system message>

<user message>
  Here is the query to be answered:
  <query>
  {query}
  </query>

  Below is the context for answering the query:
  <context>
  {context}
  </context>
</user message>
```

B Detailed Experiment Results

B.1 Performance Metrics

Table 5 reports the detailed performance of HLTM and all baselines across five backbone models. HLTM achieves the best correctness on summary-style queries and the best F1 on retrieval-style queries in every model column, with most gains over the strongest baseline statistically significant under a Bonferroni-corrected Wilcoxon test. This indicates that HLTM provides robust, model-agnostic quality improvements rather than relying on a specific LLM backbone.

Table 5: Performance metrics across models. Bold: best method per column. Underline: best baseline method per column. Stars: Wilcoxon signed-rank test of HLTM vs. best baseline, Bonferroni-corrected (* $p < 0.05$, ** $p < 0.01$, * $p < 0.001$). ↑ indicates higher is better. SimpleMem and GraphRAG omitted due to excessive latency (see Table 6).**

Method	Summary-style (Correctness ↑)					Retrieval-style (F1 ↑)				
	GPT-4o mini	Gemma 3 27B	Qwen3 32B	GLM-4 32B	GLM-4.5 Air (106B)	GPT-4o mini	Gemma 3 27B	Qwen3 32B	GLM-4 32B	GLM-4.5 Air (106B)
FULL-CONTEXT	0.791	0.629	0.677	0.638	0.735	0.544	0.440	0.451	0.422	0.482
RAG	0.770	0.717	0.738	0.738	0.759	0.604	<u>0.648</u>	<u>0.617</u>	<u>0.581</u>	0.646
SCHEMA FILTER	0.513	0.741	0.723	0.744	0.791	0.546	0.562	0.484	0.412	0.558
A-MEM	0.779	0.739	0.789	0.774	0.779	0.549	0.514	0.517	0.475	0.581
HIPPO RAG	<u>0.833</u>	<u>0.801</u>	<u>0.819</u>	<u>0.830</u>	<u>0.841</u>	0.560	0.589	0.549	0.480	0.535
RAPTOR	0.715	0.717	0.712	0.730	0.722	0.348	0.505	0.535	0.408	0.516
MEM0	0.521	0.475	0.613	0.473	0.501	0.369	0.368	0.336	0.285	0.366
READAGENT	0.796	0.744	0.762	0.785	0.813	<u>0.617</u>	0.575	0.518	0.485	<u>0.730</u>
HLT M (ours)	0.892***	0.850***	0.874***	0.867*	0.893***	0.782***	0.722***	0.751***	0.674***	0.833***

B.2 Deployment Metrics

Table 6 compares HLTM’s deployment efficiency against baselines on online query latency, LLM invocation count, and token consumption. HLTM keeps low query latency on both query types with a moderate token budget, which is substantially below full-context prompting, and lower or competitive against graph-based and other memory systems (GraphRAG, SimpleMem, HippoRAG, ReadAgent) despite using two LLM calls per query. This combination of high answer quality and bounded cost makes HLTM practical for latency-sensitive production deployment.

Table 6: Deployment efficiency metrics across query types. Left: summary-style; right: retrieval-style. Metrics include latency, LLM calls, and token consumption. Mean ± SEM over queries. ↓ indicates lower is better.

Method	Summary-style			Retrieval-style		
	Latency (s) ↓	LLM Calls ↓	Tokens (k) ↓	Latency (s) ↓	LLM Calls ↓	Tokens (k) ↓
FULL-CONTEXT	7.94 ± 0.22	1.00 ± 0.00	40.79 ± 1.17	10.23 ± 0.35	1.00 ± 0.00	37.95 ± 1.61
RAG	3.30 ± 0.13	1.00 ± 0.00	2.53 ± 0.02	3.55 ± 0.14	1.00 ± 0.00	2.70 ± 0.03
SCHEMA FILTER	3.55 ± 0.15	1.58 ± 0.02	7.62 ± 0.43	3.55 ± 0.15	1.67 ± 0.02	7.29 ± 0.51
A-MEM	4.58 ± 0.13	1.00 ± 0.00	7.21 ± 0.17	5.30 ± 0.17	1.00 ± 0.00	6.67 ± 0.18
HIPPO RAG	6.87 ± 0.14	1.95 ± 0.01	11.40 ± 0.18	10.03 ± 0.29	1.94 ± 0.02	10.97 ± 0.25
RAPTOR	2.26 ± 0.05	1.00 ± 0.00	0.53 ± 0.00	2.46 ± 0.06	0.93 ± 0.01	0.63 ± 0.01
GRAPH RAG	11.11 ± 0.16	15.86 ± 0.26	30.30 ± 0.52	7.98 ± 0.25	15.20 ± 0.43	23.11 ± 0.69
SIMPLE MEM	14.50 ± 0.15	7.01 ± 0.06	6.78 ± 0.13	17.80 ± 0.19	7.44 ± 0.06	8.76 ± 0.17
MEM0	2.24 ± 0.05	1.00 ± 0.00	0.28 ± 0.01	1.99 ± 0.04	1.00 ± 0.00	0.30 ± 0.00
READAGENT	6.51 ± 0.18	2.00 ± 0.00	11.67 ± 0.32	7.24 ± 0.28	2.00 ± 0.00	13.40 ± 0.49
HLT M (ours)	3.01 ± 0.06	2.00 ± 0.00	3.91 ± 0.05	3.46 ± 0.07	2.00 ± 0.00	4.26 ± 0.07

C Privacy Isolation Evaluation

To empirically validate HLTM’s tenant-scoped memory architecture, we conducted leakage experiments on LinkedIn’s Hiring Assistant dataset. We indexed each memory system with a cross-tenant dataset comprising hiring projects from multiple recruiter seats and contracts, then queried each system with natural-language tenant-scope constraints. We measure leakage on two dimensions: (i) **Query-wise Leakage Ratio**: the fraction of queries returning at least one out-of-scope hiring project, (ii) **Entity-wise Leakage Ratio**: the average fraction of returned entities that are out-of-scope. As shown in Table 7, baseline methods exhibit substantial leakage, with RAPTOR and GraphRAG reaching 90%. HLTM achieves 0% leakage on both metrics, owing to its hierarchical tenant-scoped indexing that structurally enforces isolation boundaries rather than relying on post-hoc filtering.

Table 7: Privacy leakage rates across memory systems on a cross-tenant hiring dataset. ↓ indicates lower value is better. HLTM achieves perfect isolation on both metrics.

Method	Query-wise Leakage ↓	Entity-wise Leakage ↓
RAG	64.0%	54.6%
SCHEMA FILTER	30.0%	36.2%
A-MEM	58.0%	48.5%
HIPPO-RAG	52.0%	42.7%
RAPTOR	90.0%	86.3%
GRAPH-RAG	92.0%	98.5%
SIMPLEMEM	30.0%	69.8%
MEM0	78.0%	69.4%
READAGENT	88.0%	77.0%
HLTМ (ours)	0.0%	0.0%

D Incremental Indexing Evaluation

The lossless nature of HLTM’s incremental updates follows directly from its tree-structured design, which exploits the independence between a node and its subtree siblings — updating one branch never affects others. We evaluated this property on LinkedIn’s complete hiring dataset by comparing two indexing strategies: (a) *full indexing*, which rebuilds the entire tree from scratch, and (b) *incremental indexing*, which removes stale leaf nodes, appends newly created ones, and refreshes updated ones before propagating changes up their ancestor paths.

We ran both strategies in parallel over two weeks on real production data, measuring alignment between the two resulting indices in terms of both memory content and tree topology. The outputs were found to be identical and fully overlapping (accounting for LLM non-determinism), directly confirming losslessness. Beyond correctness, incremental indexing reduces end-to-end latency from over 7 days to approximately 12 hours, enabling frequent re-indexing and ensuring production data freshness. Having been deployed in the production pipeline for more than six months, it achieves an **80–90% reduction in LLM invocations and cost** with no measurable degradation in output structure or memory quality.

Table 8: Incremental indexing evaluation on LinkedIn’s production dataset.

Metric	Full	Incremental
Indexing Latency	~7 days	~12 hrs
LLM call reduction	—	80–90%
Topology overlap	—	identical
Index quality	—	no degradation

E Notation Summary

Table 9: Notation summary.

Symbol	Meaning
$\mathcal{T} = (V, E)$	HLTМ rooted tree index.
$V = \{v\}$	All nodes v (business entities).
$E = \{(p, c)\}$	Directed parent–child edges.
$\mathcal{T}_v$	Subtree rooted at v (privacy / access scope).
D_v	Raw documents attached to v .
$M_v = (F_v, Q_v, S_v)$	Memory triple at node v ; $M_v = \mathcal{M}(v)$.
F_v	Facet view: salient key–value pairs at v .
Q_v	QA view: question–answer pairs at v .
S_v	Summary view: paragraph + one-sentence summary at v .
q	User query text.
F_q	Facet set parsed from q .
$s_{\text{facet}}, s_{\text{QA}}, s_{\text{summary}}(q, v)$	Per-view retrieval scores for node v .
$k_{\text{facet}}, k_{\text{QA}}, k_{\text{summary}}$	Top- k cutoffs for each view.
$F_k(q), Q_k(q), S_k(q)$	Top- k memories from facet / QA / summary views.
V^*	Leaf nodes updated since last indexing cycle.
$\text{emb}(\cdot)$	Text-to-vector embedding function.
$\text{LLM}(q, \text{ctx})$	Answer generation conditioned on q and retrieved memories.

Received 8 February 2026; revised 17 April 2026; accepted 16 May 2026